Solution: Edit Distance

Problem Statement

Given strings s1 and s2, we need to transform s1 into s2 by deleting, inserting, or replacing characters. Write a function to calculate the count of the minimum number of edit operations.

Example 1:

```
Input: s1 = "bat"
       s2 = "but"
Output: 1
Explanation: We just need to replace 'a' with 'u' to transform s1 to s2.
```

Example 2:

```
Input: s1 = "abdca"
       s2 = "cbda"
Output: 2
Explanation: We can replace first 'a' with 'c' and delete second 'c'.
```

Example 3:

```
Input: s1 = "passpot"
       s2 = "ppsspqrt"
Output: 3
Explanation: Replace 'a' with 'p', 'o' with 'q', and insert 'r'.
```

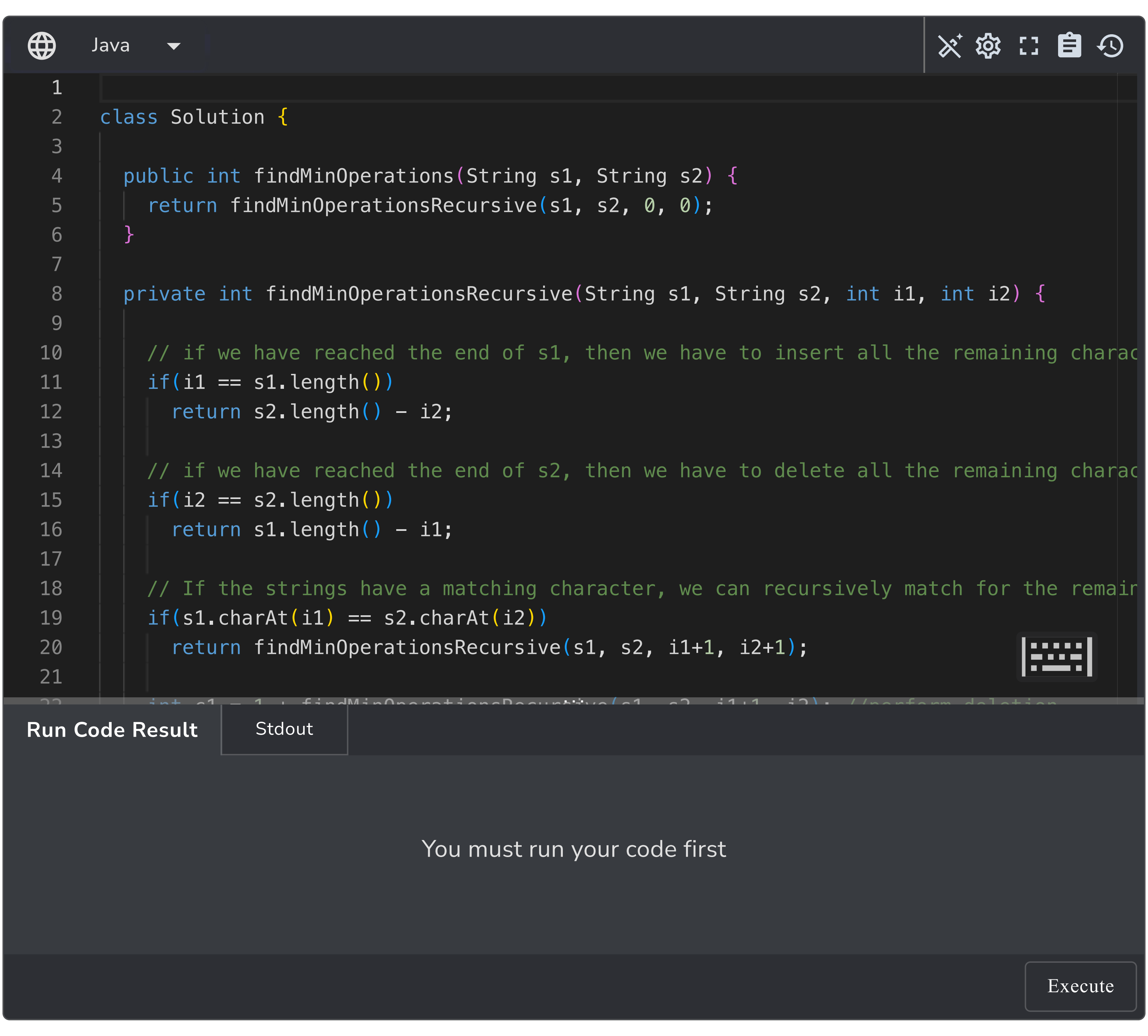
Basic Solution

A basic brute-force solution could be to try all operations (one by one) on each character of s1. We can iterate through s1 and s2 together. Let's assume index1 and index2 point to the current indexes of s1 and s2 respectively, so we have two options at every step:

1. If the strings have a matching character, we can recursively match for the remaining lengths.
2. If the strings don't match, we start three new recursive calls representing the three edit operations. Whichever recursive call returns the minimum count of operations will be our answer.

Code

Here is the recursive implementation:



```

class Solution {

  public int findMinOperations(String s1, String s2) {
    return findMinOperationsRecursive(s1, s2, 0, 0);
  }

  private int findMinOperationsRecursive(String s1, String s2, int i1, int i2) {

    // if we have reached the end of s1, then we have to insert all the remaining charac
    if(i1 == s1.length())
      return s2.length() - i2;

    // if we have reached the end of s2, then we have to delete all the remaining charac
    if(i2 == s2.length())
      return s1.length() - i1;

    // If the strings have a matching character, we can recursively match for the remair
    if(s1.charAt(i1) == s2.charAt(i2))
      return findMinOperationsRecursive(s1, s2, i1+1, i2+1);

```

Because of the three recursive calls, the time complexity of the above algorithm is exponential $O(3^{m+n})$, where 'm' and 'n' are the lengths of the two input strings. The space complexity is $O(n+m)$ which is used to store the recursion stack.

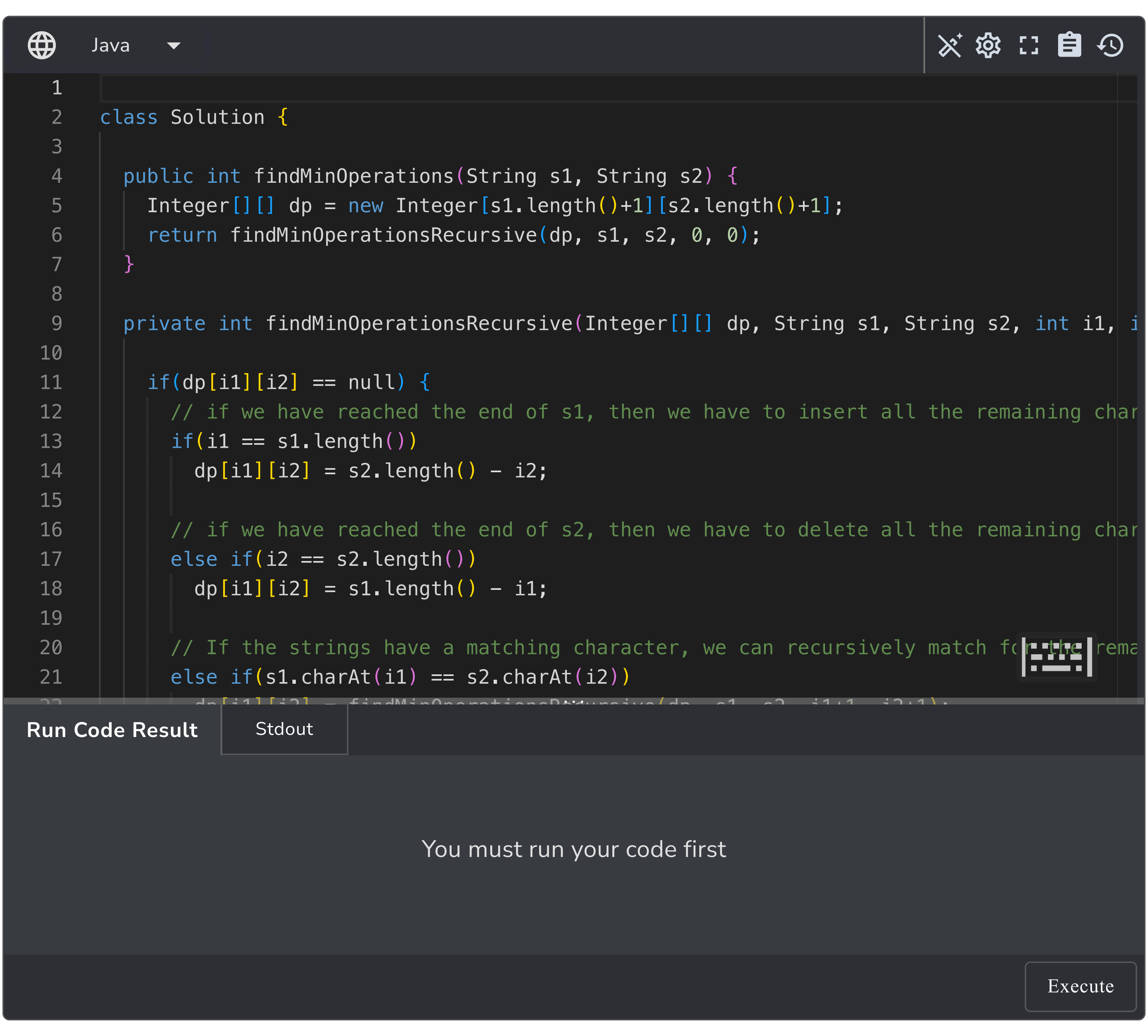
Top-down Dynamic Programming with Memoization

We can use an array to store the already solved subproblems.

The two changing values in our recursive function are the two indexes, i1 and i2. Therefore, we can store the results of all the subproblems in a two-dimensional array. (Another alternative could be to use a hash-table whose key would be a string (i1 + "|" + i2)).

Code

Here is the code for Top-down DP approach:



```

class Solution {

  public int findMinOperations(String s1, String s2) {
    Integer[][] dp = new Integer[s1.length()+1][s2.length()+1];
    return findMinOperationsRecursive(dp, s1, s2, 0, 0);
  }

  private int findMinOperationsRecursive(Integer[][] dp, String s1, String s2, int i1, i

    if(dp[i1][i2] == null) {
      // if we have reached the end of s1, then we have to insert all the remaining char
      if(i1 == s1.length())
        dp[i1][i2] = s2.length() - i2;

      // if we have reached the end of s2, then we have to delete all the remaining char
      else if(i2 == s2.length())
        dp[i1][i2] = s1.length() - i1;

      // If the strings have a matching character, we can recursively match fo        rema
      else if(s1.charAt(i1) == s2.charAt(i2))
```

What is the time and space complexity of the above solution? Since our memoization array `dp[s1.length()][s2.length()]` stores the results for all the subproblems, we can conclude that we will not have more than $m*n$ subproblems (where 'm' and 'n' are the lengths of the two input strings.). This means that our time complexity will be $O(m*n)$.

The above algorithm will be using $O(m*n)$ space for the memoization array. Other than that we will use $O(m+n)$ space for the recursion call-stack. So the total space complexity will be $O(m*n+(m+n))$, which is asymptotically equivalent to $O(m*n)$.

Bottom-up Dynamic Programming

Since we want to match all the characters of the given two strings, we can use a two-dimensional array to store our results. The lengths of the two strings will define the size of the two dimensions of the array. So for every index 'i1' in string 's1' and 'i2' in string 's2', we will choose one of the following options:

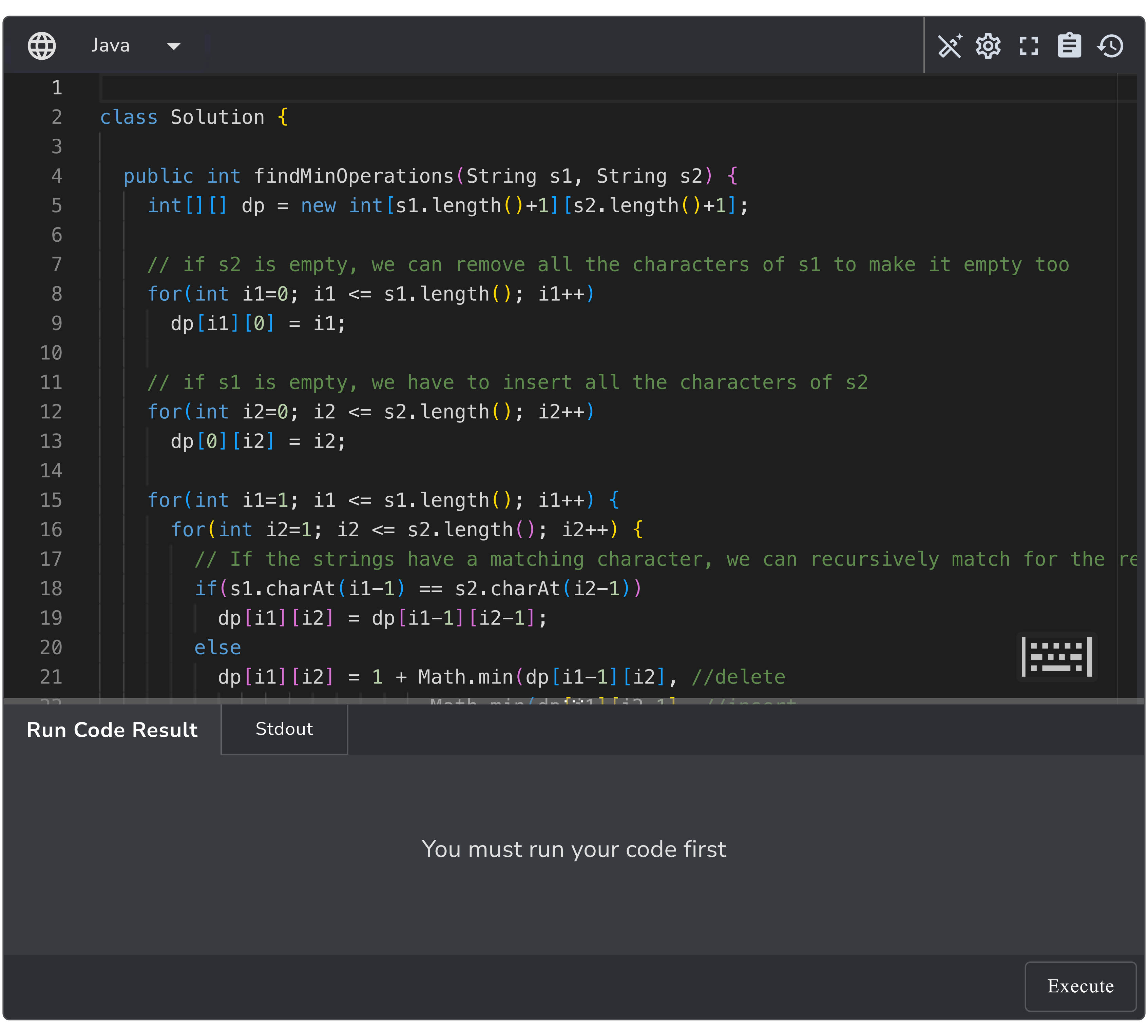
1. If the character `s1[i1]` matches `s2[i2]`, the count of the edit operations will be equal to the count of the edit operations for the remaining strings.
2. If the character `s1[i1]` does not match `s2[i2]`, we will take the minimum count from the remaining strings after performing any of the three edit operations.

So our recursive formula would be:

```
if s1[i1] == s2[i2]
  dp[i1][i2] = dp[i1-1][i2-1]
else
  dp[i1][i2] = 1 + min(dp[i1-1][i2], // delete
                      dp[i1][i2-1], // insert
                      dp[i1-1][i2-1]) // replace
```

Code

Here is the code for our bottom-up dynamic programming approach:



```

class Solution {

  public int findMinOperations(String s1, String s2) {
    int[][] dp = new int[s1.length()+1][s2.length()+1];

    // if s2 is empty, we can remove all the characters of s1 to make it empty too
    for(int i1=0; i1 <= s1.length(); i1++)
      dp[i1][0] = i1;

    // if s1 is empty, we have to insert all the characters of s2
    for(int i2=0; i2 <= s2.length(); i2++)
      dp[0][i2] = i2;

    for(int i1=1; i1 <= s1.length(); i1++) {
      for(int i2=1; i2 <= s2.length(); i2++) {
        // If the strings have a matching character, we can recursively match for the re
        if(s1.charAt(i1-1) == s2.charAt(i2-1))
          dp[i1][i2] = dp[i1-1][i2-1];
        else
          dp[i1][i2] = 1 + Math.min(dp[i1-1][i2], //delete
```

The time and space complexity of the above algorithm is $O(n*m)$, where 'm' and 'n' are the lengths of the two input strings.

← Previous
Edit Distance

Next →
Strings Interleaving

☐ Mark as Completed